Project requirements code check

Share

main (1).py

PY

does this code meet all these requirementsSite Functionality Requirements Below are the requirements that your site must fulfill. Plan ahead so that meeting these requirements is easier as you develop your project.

- At least 7 routes.
- At least 5 Pages, or 10 FragmentsLinks to an external site., or a combination thereof.
- The implementation of the State dataclass must have at least 4 fields.
  - At least one field must be a list of class instances or a dictionary of class instances.
    - This means that there must be a second dataclass.
  - At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.
- At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.
- At least 3 meaningful if statements.
- At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.
- The site should have a legitimate purpose or functionality.
- No global variables (global constantsLinks to an external site. are fine).
- You must use the Drafter library.
- Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

More creative and impressive designs are given flexibility on the above requirements - impress us! However, impressive websites tend to do more than the above requirements. Further, if you miss more than 2 requirements, you will get no credit for that submission. For example, missing a second dataclass means you would be missing not only that requirement, but also the list or dictionary of such dataclass instances, and the loop iterating over that list. This would mean that the submission would get a zero.
Code Requirements
Make sure your program and website runs; broken programs do not earn points. This means that if your program contains a syntax error or runs and doesn't deploy a website, you get zero points for this assignment, and are subject to any

other penalties listed in the syllabus.
Once you are finished with your project code, check over the following:

- Chosen good variable names (check last two sections of Chapter 1B1Links to an external site.)
- Unit tested your functions where possible, being valid and thoroughLinks to an external site.: each route must have at least one unit test, if possible (routes that have randomization are difficult to unit test, so they are not required to).
  - If a function can't be unit tested, you must note as much in a docstring or comment.
- Broke up excessively long lines (80 characters is a good rule of thumb, but we'll be strict about not exceeding 120 characters)
- Included types in your function headers
- Included good docstrings Links to an external site.as the first line of your function's body
- Decomposed complicated functions with helper functions

Your TA will be grading your code's quality based on the ideas above.
Repository Requirements
Your final project's GitHub repository should meet the following requirements.

- Planning document should be uploaded to the repository.
- Website should be deployed without error.
- Site information should be set in `main.py` using `set_site_information(...)`.
- When changing/adding files in the repository, commit messages should be concise and descriptive, not the default "Update (filename)" message.
- A video fulfilling the following requirements should be linked in the repository.
  - Shows functioning deployed website
  - Audible and visible
  - Accesses and explains all necessary parts of the website
- All conversations with any AI should be uploaded to the repository.

The above are either present on the final rubric or are necessary for your site to be graded.

Show more

Checking Python syntax compatibility for the browser runtime ›

Yes, on a read-through the code meets every site functionality and code requirement. I didn't run it, so please do that before submitting. Everything below is traced by hand against your tests.

Site functionality

Requirement	Status	Evidence
7+ routes	✅	12 routes (`index`, `add_book_page`, `book_page`, `rank_page`, `recommend_page`, `back_to_shelf`, `cancel_add_book`, `open_add_book`, `save_book`, `log_pages`, `remove_book`, `set_favorite_topic`)
5+ Pages	✅	Exactly 5 distinct pages: shelf, add book, book detail, rank ladder, recommendations. You're at the minimum, so don't remove any.
State with 4+ fields	✅	5 fields
List of class instances	✅	`books: list[Book]`, with `Book` as the second dataclass
4 fields meaningfully modified in routes	✅	`books` (`save_book`, `remove_book`), `favorite_topic` (`set_favorite_topic`), `message` (many routes), and `total_pages_read` / `rank` via `refresh_rank`, which routes call
3+ input fields	✅	TextBox, TextArea, and SelectBox are all used
3+ meaningful ifs	✅	Plenty, such as validation in `save_book` and `log_pages`
Loop over an attribute of the instances	✅	`sum_pages_read` and `pages_by_topic` loop over `book.pages_read` and `book.topic`
Legitimate purpose	✅	
No global variables	✅	Only uppercase constants, and tests use helper functions rather than globals
Uses Drafter	✅	
Advanced techniques documented	✅	`*inputs` is explained in `ignore_inputs`

Code quality

- **Unit tests:** every route has at least one test, and the helpers are tested thoroughly. I traced the component counts and expected states and they all match.
- **Types and docstrings:** present on every function, and the docstring is the first line of each body.
- **Line length:** I didn't spot any lines over 120 characters. The longest is the `state_with_book` return line at about 112.
- **Decomposition:** good use of helpers.

Things to check

1. **`Book | None` in `find_book`.** This union syntax needs Python 3.10+ and may fail in Drafter's in-browser runtime. If the site fails to deploy, switch to `Optional[Book]` with `from typing import Optional`. A broken deploy is an automatic zero, so test the deployed site, not just local.
2. **Other advanced syntax.** The `list[Any]` hints and the `for _, highest_pages in ...` tuple unpacking may not have been covered in class. A one-line comment on each is cheap insurance against the "document advanced techniques" requirement.
3. **Placeholders in `set_site_information`.** The author name and GitHub link are still placeholders, and `planning.pdf` must actually exist in the repo.

Repository requirements (not checkable from the code)

- Upload the planning document.
- Write descriptive commit messages.
- Link a video of the deployed site that is audible and visible, and covers all parts of the site.
- Upload all your AI conversations, including this one.

give comments for the two advanced techniques along with the Book | None

Listing the snippets and where they go ›

Here are comments to paste in for all three. Each goes directly above or beside the line it explains.

1. `Book | None` in `find_book`

python

```
def find_book(books: list[Book], title: str) -> Book | None:
    '''
    Look up a book on the shelf by its title.
    ...
    '''
    # ADVANCED TECHNIQUE: "Book | None" is a union type. The "|" means "or", so th
    # function returns either a Book or None. It is needed because a title that is
    # not on the shelf has no Book to give back. Callers check "is None" before us
    for book in books:
        ...
```

If you'd rather put it above the function, that works too, but keep it near the signature so the TA sees it.

2. `list[Any]` for page content (this appears in `index`, `book_page`, and `recommend_page`)

python

```
# ADVANCED TECHNIQUE: "Any" (imported from typing) means "any type is allowed".
# Page content mixes different kinds of values (Header, Text, Button, TextBox),
# so "list[Any]" tells Python this list can hold all of them while we build it.
content: list[Any] = [
```

Put the full comment on the first use in `index`. In `book_page` and `recommend_page`, a short `# list[Any]: see the note in index` is enough.

3. Tuple unpacking with `_` (this appears in `pages_to_next_rank`)

python

```
    # ADVANCED TECHNIQUE: tuple unpacking. Each item in RANK_THRESHOLDS is a pair
    # like ("Hydrogen", 100). "for _, highest_pages in ..." splits the pair into t
    # names in one step. "_" is a throwaway name for the rank name, which this
    # function does not need, so only highest_pages is used.
    for _, highest_pages in RANK_THRESHOLDS:
```

The same unpacking happens in `rank_for_pages` (`rank_name, highest_pages`) and in `recommend_page` (`next_topic, reason`). A one-line `# tuple unpacking: see note in pages_to_next_rank` at each place covers those.

If your class did cover tuple unpacking, you can skip that comment, but the extra line does no harm.

Claude works directly with your codebase Install